Analisis de datos en python

Gino Anci

20 de marzo de 2026

Índice general

1 Introducción **2**

2 NumPy **3**

2.1 Introdución 3
2.1.1 Arreglos multidimensionales: Más allá de las listas . 3
2.1.2 Array 5
2.1.3 ¿Por qué usar NumPy? 6
2.2 Instalación 6
2.3 Primeros pasos 7
2.3.1 Array simple 7
2.4 Creación de Arrays 12
2.4.1 Arrays desde listas o tuplas 12
2.4.2 Función *np.arange* 13
2.4.3 Función *np.linspace* 14
2.4.4 Función *np.zeros* 15
2.4.5 Función *np.ones* 16
2.4.6 Función *np.eye* 17
2.5 Propiedades y estructura de los arrays 18
2.5.1 Dimensiones, forma, tamaño y tipo de datos 18
2.5.2 Conversión de tipos *.astype()* 24
2.5.3 Indexación y slicing 25
2.6 Operaciones matemáticas 31
2.6.1 Raiz cuadrada y exponencial 31
2.6.2 Logaritmo Natural 32
2.6.3 Funciones trigonométricas 33

Capítulo 1

Introducción

Este documento ha sido diseñado como una guía técnica orientada específicamente a la implementación de código para el análisis de datos. A diferencia de los manuales convencionales, este material no es una introducción a Python. Se asume que el lector posee un dominio fluido de la sintaxis básica, estructuras de control y tipos de datos del lenguaje.

El objetivo aquí no es enseñar a programar, sino transformar la habilidad de programación en una herramienta de exploración científica. Nos enfocaremos en la manipulación eficiente de estructuras multidimensionales, la limpieza técnica de datasets y la traducción de preguntas de negocio en implementaciones algorítmicas utilizando el ecosistema estándar de la industria (Pandas, NumPy, Matplotlib).

Capítulo 2

NumPy

2.1 Introdución

NumPy (*Numerical Python*) es una libreria de *Python* que se *especializa* al manejo de *arreglos multidimensionales (arrays)* y en la realización de cálculos numéricos de alto rendimiento.

Para el analista, entender NumPy no es solo aprender una librería, sino adoptar una nueva forma de pensar la información "el pensamiento matricial".

2.1.1 Arreglos multidimensionales: Más allá de las listas

¿A qué nos referimos exactamente con "multidimensional"? En términos computacionales, un arreglo multidimensional es una estructura de datos que organiza la información en múltiples ejes o direcciones.

Aunque matemáticamente podemos trabajar con n dimensiones, para facilitar la intuición técnica, solemos categorizarlas en las tres arquitecturas más comunes:

- Unidimensional: Conocido como *Vector*.
- Bidimensional: Conocido como *Matriz*.
- Tridimensional: Conocido como *Tensor*

Una dimensión (Vector)

Vector no es el nombre de tu amigo (o tal vez sí), pero para lo que nos ocupa, es simplemente una lista de elementos dispuestos en una sola

dirección. Matemáticamente, un vector se ve así:

$$V = [x_0, x_1, x_2, \ldots, x_{n-1}]$$

Ahora bien, a usted lector, probablemente no le interese en lo más mínimo cómo se ve esto en un libro de álgebra lineal; si así fuera, seguramente estaría leyendo álgebra lineal en lugar de este apunte. Para simplificarle la vida, imagine a un vector como una *fila de la escuela*: esa donde el más bajo iba adelante y el más alto atrás. O una fila de asientos en el cine.

Un vector puede representarse como una fila o como una columna de datos. Suponga que tiene una lista de nombres donde cada uno representa a una persona; en este caso, tenemos un *vector de nombres*.

Lo fundamental aquí es que un vector es una estructura unidimensional. No importa si los datos están acostados o parados: solo necesitamos un único *índice* (i) para saber exactamente en qué posición se encuentra un objeto. Si usted sabe que "Juan" es el tercero de la fila, no necesita más coordenadas para encontrarlo. En *Python*, ese "tercer lugar" es lo único que nos importa para extraer el dato.

Dos dimensiones (Matriz)

Una *matriz* es, para fines prácticos, una tabla de filas y columnas. Si usted ha sobrevivido a una hoja de *Excel* o ha intentado jugar un partido de ajedrez, ya conoce esta estructura. Aquí los datos dejan de ser una simple fila india para organizarse en dos direcciones: filas (i) y columnas (j).

Matemáticamente, una matriz se ve de la siguiente manera:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \cdots & \cdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

Nuevamente, no se asuste con los subíndices; solo están ahí para indicar que ahora necesitamos dos coordenadas para encontrar un dato. Si estuviéramos en un tablero de ajedrez y quisiéramos ubicar a la reina blanca, diríamos que está en la posición $1, d$. En el mundo de los datos, la lógica es idéntica: primero miramos la fila y luego la columna.

Sin embargo, hay una trampa en la que el programador novato suele caer: en *Python* (y en casi toda la computación moderna), el primer elemento no vive en la posición 1, sino en la 0. Por lo tanto, para acceder al primer dato de nuestra "tabla", utilizaremos la sintaxis [0,0]. Es un pequeño precio a pagar por la eficiencia, aunque a su cerebro le tome un par de días acostumbrarse a no empezar a contar desde el uno.

Tres dimensiones (Tensor)

Para no complicarle la vida (ni complicármela yo), imagine un *tensor* simplemente como un cubo o una pila de matrices. Si una matriz es una hoja de cálculo, un tensor es el libro entero de *Excel* donde cada pestaña es una capa de datos. Para ubicar un elemento aquí, ahora necesitamos tres coordenadas: el *piso* (o profundidad), la *fila* y la *columna*.

Básicamente, el mundo físico que habitamos es de tres dimensiones, así que la intuición no debería fallarle.

Tal como usted, no pienso perder el tiempo mostrando cómo se ve un tensor representado matemáticamente; sospecho que ambos tenemos mejores cosas que hacer. Pero si necesita una imagen mental, piense en un *cubo de Rubik*: para identificar un cuadradito específico, usted debe saber en qué cara está (profundidad), en qué fila se encuentra y en qué columna se cruzan.

En el código, esto se traduce en una terna de índices $[z, i, j]$. Donde z representa el nivel o la capa, e i y j son las coordenadas de la "tabla" que ya conocimos. Si logra visualizar esto, felicidades, acaba de entender la estructura base que utilizan las redes neuronales modernas para procesar imágenes a color (donde cada "piso" es un canal: Rojo, Verde y Azul).

2.1.2 Array

En programación, un *array* (o "arreglo", si es que usted todavía se resiste al inglés técnico) es una colección de elementos, generalmente números, organizados de una forma específica.

El origen de la palabra, aunque probablemente no le interese, proviene del francés antiguo *arrai*, que significa "poner en orden" o "disponer". Si recupera por un breve instante lo que acaba de leer sobre arreglos multidimensionales, entenderá el punto: los elementos no se tiran al azar dentro de la memoria de la computadora.

Cada dato tiene una posición fija, un *índice* que nos permite encontrarlo rápidamente sin tener que revolver toda la caja. En *NumPy*, esta "disposición" es sagrada. A diferencia de las listas de *Python* convencionales, que son flexibles y un tanto desordenadas, un *array* de *NumPy* es una estructura rígida, eficiente y perfectamente indexada. Si usted sabe el índice, tiene el poder; si no lo sabe, está perdido en la memoria del sistema.

2.1.3 ¿Por qué usar NumPy?

Si usted quiere simplificarse la vida y no dejar a su computadora agonizando mientras consume ciclos de CPU innecesarios, le recomiendo encarecidamente que preste atención.

NumPy no es solo una opción, es el estándar porque permite:

- *Eficiencia a escala:* Trabajar con grandes volúmenes de datos de forma masiva, optimizando el uso de la memoria RAM.
- *Velocidad pura:* Los *arrays* son significativamente más rápidos que las listas de *Python*, ya que almacenan los datos de forma contigua en la memoria (algo que a su procesador le encanta).
- *Vectorización:* Incluye operaciones que actúan sobre bloques enteros de datos, permitiéndonos jubilar esos bucles *for* lentos y visualmente espantosos.
- *Caja de herramientas científica:* Contiene funciones matemáticas avanzadas, herramientas de álgebra lineal, estadística y generadores de números aleatorios.
- *Y mucho más:* No voy a nombrar cada beneficio; *NumPy* no me paga por publicitarlo y, siendo honestos, si no fuera software libre, probablemente me cobrarían.

2.2 Instalación

Voy a suponer que usted no ha llegado a este apunte sin haber instalado antes otras librerías. En el caso de que sea la primera vez que se enfrenta a la instalación de paquetes, intentaré ser claro (aunque, por favor, espero que ya lo haya hecho antes).

Básicamente, para instalar *NumPy*, al igual que *Pandas* o *Matplotlib*, el procedimiento es el mismo. Dado que asumo que si algún lector saltó directamente a las otras secciones ya dominaba este arte, esta será la única vez que explicaré cómo gestionar paquetes.

Para realizar la instalación, diríjase a la terminal de su editor de código —o si escribe su código directamente en la terminal creyéndose *Linus Torvalds* (en cuyo caso no deberia estar leyendo esto)— y ejecute el siguiente comando:

```
pip install numpy
```

Puede, a su vez, reemplazar *numpy* por *pandas* o *matplotlib* para descargar el resto del ecosistema. Supongo que sabe que es recomendable realizar estas instalaciones dentro de un *entorno virtual*, pero no voy a explicar aquí qué es eso o cómo se configura. Quizás algún día dedique otro apunte a esos menesteres; por ahora, asumo que su entorno está listo para la acción.

2.3 Primeros pasos

Lamentablemente, es hora de dejar la tan hermosa sección de introducción. ¡La voy a extrañar, introducción! Pero como aquí vinimos a procesar datos y no a escribir poesía, es momento de ensuciarnos las manos.

Antes de empezar como locos, hay que importar la librería, porque de lo contrario Python no tendrá idea de qué estamos hablando. Para esto, utilizamos las palabras mágicas: *Import numpy as np*. Realmente, la parte de *as np* no es estrictamente obligatoria, pero a la hora de escribir código es mucho más fácil y rápido escribir dos letras que cinco. Si usted decide no usar el alias, prepárese para desperdiciar segundos valiosos de su vida escribiendo *numpy* frente a cada función.

2.3.1 Array simple

Un *array* simple es, esencialmente, el *vector* que vimos en la introducción: una secuencia lineal de datos. Sin embargo, para que *NumPy* haga su magia, este arreglo debe respetar tres reglas de oro:

- *Homogeneidad:* Todos los elementos deben ser del mismo tipo. Si un *array* es de números enteros, no puede pretender meter una palabra o un *string* en el medio. Esta rigidez es, precisamente, lo que permite que sea mucho más rápido que las listas convencionales de *Python*.

- *Índices:* Cada posición tiene un número de orden que, como ya mencionamos (y espero que no haya olvidado), empieza siempre en 0. Por lo tanto, si su *array* tiene 5 elementos, el último vive en la posición 4.

- *Tamaño fijo:* Una vez que se crea el *array* en la memoria, su tamaño queda definido. Aunque existen formas de "alterarlo", lo que realmente sucede tras bambalinas es que se crean copias modificadas y se borran las anteriores. No es tan flexible como una lista, pero es el precio de la eficiencia.

Primer codigo

Lamentablemente, en *NumPy* nuestro primer código no va a ser el clásico e inútil "Hola mundo"; aquí nuestro bautismo de fuego es el *array simple*. Veamos cómo se traduce a código todo esto que venimos hablando:

Creación y acceso en un array 1D

```python
import numpy as np

# Estaturas al azar; el autor no tenía mucha creatividad hoy.
estaturas = np.array([1.75, 1.82, 1.60, 1.91])

# Accediendo al primer elemento.
primero = estaturas[0]  # Resultado: 1.75

# Si quisiéramos saber cuántos elementos tiene el arreglo
cantidad = estaturas.size # Resultado: 4
```

Ahora usted me dirá: "para eso uso una lista de toda la vida". Pero recuerde, *NumPy* está hecho para cuando tengamos un millón de estaturas y queramos procesarlas en milisegundos. Por cuestiones *didácticas*, no voy a realizar una toma de un millón de datos en este ejemplo; sería una crueldad innecesaria para su memoria RAM. Además, usted lector no se tomaría el trabajo de leer una muestra de ese tamaño, probablemente cerraría este apunte y se iría a ver una serie. Aquí lo que importa es que entienda la lógica, no que vea números infinitos.

La regla de la homogeneidad

Como se menciono anteriormente un array tiene todos los espacios contados, si intentaramos meter algo diferente, *NumPy* va a tener que tomar una decisión para que el array siga siendo eficiente.

Usted lector, curioso y quizás algo malvado por naturaleza, seguramente se cuestiona: ¿Qué pasa si intento romper la regla adrede? Si usted crea un *array* mezclando números enteros o flotantes con palabras, *NumPy* transformará a *todos* los elementos al tipo de dato más flexible para que ninguno pierda información. En la jerarquía de datos, esto suele significar que todo terminará convertido en un *string*.

Un Intruso en el array

```python
import numpy as np

edades = np.array([25, 30, 'Gino', 22]) # ¡Gino no es una edad, es mi nombre!

# Imprimimos edades
print(edades) # devuelve ['25' '30' 'Gino' '22']
```

Si usted está atento, se habrá dado cuenta de que *NumPy* puso comillas en todos los valores. Básicamente, convirtió a sus pobres números en simples textos.

NumPy hace esto para salvarle las papas a su computadora. Recuerde que la librería asigna un espacio fijo en la memoria para cada elemento; si todos son enteros pequeños, reserva pocos *bytes*, pero si aparece un *string*, debe asignar espacio suficiente para el texto más largo. Para mantener la estructura "ordenada" que vimos antes, todos deben medir lo mismo.

Este proceso de conversión automática se denomina *coerción de tipos*, aunque probablemente usted y yo lo olvidemos apenas cerremos este apunte.

Operaciones vectorizadas

Esta es la razón por la cual usamos *Numpy* y no otra calculadora o una lista de python.

La *vectorización* es la capacidad de aplicar una operación matemática a un *array* completo, o entre dos *arrays*, sin necesidad de escribir ciclos (*loops*) manuales. Para entenderlo de forma sencilla: imagine que tiene que firmar y poner sus datos en 100 hojas. Puede hacerlo a mano con una lapicera (listas normales), o puede mandar a fabricar un sello con todos sus datos y simplemente presionar el mismo sobre cada hoja. *NumPy* es el sello.

Ejemplo 2.3.1

Imagine que usted es un chef y una pareja que va a casarse lo contrata para servir 1.000 platos de fideos; además, a cada uno debe ponerle una montaña de queso rallado.
Si usara *Python* normal, usted debería ir con un rallador y un bloque de queso al primer plato: ralla, ralla, ralla. Luego se dirige al segundo: ralla, ralla, ralla... y así hasta el plato 1.000. El resultado: para cuando termine, los fideos estarán fríos, usted estará viejo y la boda

> probablemente habrá terminado hace décadas.
> Pero como usted es un chef inteligente, le pide a la pareja que gaste 60.000 dólares en una máquina industrial con un sistema de ventilación en el techo, cargado con 50 kg de queso. Usted solo aprieta un botón y una ráfaga de aire precisamente calculada deposita el queso en los 1.000 platos al mismo tiempo.
> Lamentablemente, aunque la idea sea buena, probablemente al chef no lo vuelvan a contratar o lo llamen loco. Pero nosotros no somos chefs, somos analistas de datos. *NumPy* es esa máquina industrial: hace el trabajo sucio por nosotros en un solo paso, mientras que de forma manual nos tomaría una eternidad.

Como aún no hemos avanzado lo suficiente como para escribir un código complejo, veamos cómo se ve esta "magia" en un ejemplo sencillo. Una operación vectorizada se vería así:

Operación vectorizada

```python
import numpy as np

# Creamos la variable platos de fideos, donde el elemento de la lista es el peso del plato en gramos
platos = np.array([400, 412, 378, 415])

# Para cada plato rallamos 20 gramos de queso pero con la maquina automatizada sin maquina automatizada seria iterar en el bucle for
rallar_queso = platos + 20
print(f'Peso del plato sin queso = ', platos) # [400 412 378 415]

print(f'Peso del plato con queso = ', rallar_queso) # [420 432 398 435]
```

Veamos ahora que la magia no se limita a sumar un número solitario; también podemos operar entre vectores completos.

Ejemplo 2.3.2

La vida de un chef es difícil: no solo tiene que rallar queso, sino que probablemente deba lidiar con cinco platos de distintos tamaños y cinco porciones de salsa. Para obtener el peso total de cada comida, usted no quiere tirar cualquier salsa sobre cualquier pasta. Usted necesita que la salsa [0] vaya al plato [0], la salsa [1] al plato [1], y así sucesivamente.

Si quisiéramos realizar una operación entre vectores en código, *NumPy* se encarga de la logística de emparejamiento por nosotros:

Operación de vectores

```python
import numpy as np

# 1. Creamos el vector de "Pasta" (el peso en gramos de 5 platos diferentes)
pasta = np.array([100, 120, 110, 130, 105])

# 2. Creamos el vector de "Salsa" (el peso de la salsa lista para cada plato)
salsa = np.array([50, 60, 55, 65, 52])

# NumPy es inteligente: sabe que debe sumar la posición [0] con la [0], la [1] con la [1], y así hasta el final. No necesitas decirle cómo hacerlo.
plato_completo = pasta + salsa

# --- RESULTADOS ---
print("Pesos de la Pasta:      ", pasta)
print("Pesos de la Salsa:      ", salsa)
print("-----------------------------------------")
print("PESO TOTAL DE CADA PLATO:", plato_completo)
```

Sin embargo, ojalá la vida fuera todo colores y chispitas. Existe una ley inquebrantable en la aritmética de arreglos: para poder realizar una suma, el tamaño del primer vector *debe ser exactamente igual* al tamaño del segundo.

Si usted tiene cinco platos de pasta pero solo cuatro porciones de salsa, *NumPy* no va a "adivinar" qué plato se queda vacío; simplemente lanzará un error (*ValueError*) y detendrá su cocina industrial. En el caso de la multiplicación existen otras reglas de etiqueta matemática, pero eso lo

veremos más adelante, cuando sea estrictamente necesario para no saturar su paciencia.

2.4 Creación de Arrays

Hay varias formas de generar un *array*: puede ser a través de listas o tuplas, mediante funciones como *arange*, *linspace*, y más. Sin embargo, usted no está aquí para que yo simplemente se las mencione; usted las quiere ver en acción, así que vayamos a eso.

2.4.1 Arrays desde listas o tuplas

NumPy nos permite transformar las estructuras clásicas de *Python* en potentes arreglos utilizando la función que ya hemos estado "spoileando" en las secciones anteriores:

`np.array()`

Si ya tiene una lista o una tupla y desea dotarla de superpoderes para operar con ella en *NumPy*, lo único que debe hacer es pasarle ese objeto como argumento a la función. Véalo en el siguiente bloque de código:

array desde listas y tuplas

```python
import numpy as np

# ---- Caso Lista ---- #
# Tenemos una lista
lista = [1,2,3,4,5] # El autor no tenía muchas ganas de pensar nombres hoy.

# La convertimos en array
array_desde_lista = np.array(lista) # ¡Si no pense el nombre de la lista, menos el del array!

# ---- Caso Tupla ---- #
tupla = (10,20,30,40)
array_desde_tupla = np.array(tupla)

# ---- Sorpresa ---- #
lista_bidimensional = [[1,2,3], [4,5,6]] # ¡Estamos creando una matriz!
```

```
array_matriz = np.array(lista_bidimensional)
```

Para su felicidad, usted, mi lector ansioso, ahora puede ver cómo se materializa en código aquel concepto de *matriz* que explicamos en la introducción. Note cómo *NumPy* interpreta automáticamente las listas anidadas como filas de una tabla.

2.4.2 Función *np.arange*

Python nativo tiene una función llamada *range*. Pues bien, *np.arange* (que proviene de *array range*, de ahí la "a" adicional) es la versión supercargada de esa función.

Es una herramienta que crea un *array* de una sola dimensión, rellenándolo con números que siguen un orden lógico: una *progresión aritmética*.

Una progresión aritmética es simplemente una serie de números donde la distancia entre un escalón y el siguiente es siempre la misma. Matemáticamente, una progresión con paso tres (3) se vería así:

$$2, 5, 8, 11, 14$$

Más allá de la matemática, imagine a *np.arange* como una máquina que anota números en una lista, a la cual usted debe darle tres instrucciones: dónde comenzar, dónde terminar y de cuánto en cuánto avanzar. La estructura es:

```
np.arange(inicio, fin, paso)
```

Pero ojo al piojo, el número de parada *nunca* pero *nunca* se incluye en el resultado final. S le pides que pare en 10, podria llegar hasta el $9,9999\ldots$ pero nunca tocará el 10. Matemáticamente esto se veria como $[inicio, fin)$ donde el "[]" es un intervalo cerrado, es decir que los números extremos está dentro, es decir lo incluye, y el "()" es un intervalo abierto, donde uno se puede acercar mucho al valor, pero nunca tocar el borde.

Veámoslo con algunos ejemplos prácticos:

```
np.arange

import numpy as np

# Generamos una propersión aritmética solo con el
    valor donde debe finalizar (automaticamente
```

```
    comienza en 0)
arange_sin_inicio_ni_paso = np.arange(5) # si lo
    imprimieramos nos devolveria [0, 1, 2, 3, 4]

# Con inicio y fin
arange_sin_paso = np.arange(5, 10) # nos devolveria
    [5, 6, 7, 8, 9]

# Con todo incluido
arange_completo = np.arange(0,11,2) # devuelve [0, 2,
    4, 6, 8, 10]

# Con decimales
arange_decimal = np.arange(0, 1, 0.1) # devuelve
    decimales del 0.1 al 0.9
```

Usted, lector escéptico, me dirá: "¿Para qué me sirve esto en la vida real?". Pues claramente no le servirá para ir a comprar el pan, pero si usted quisiera realizar un gráfico de ventas mes a mes, podríamos utilizar esta función para generar los números de los meses del 1 al 12 sin complicaciones.

Ahora, si a usted le interesan las finanzas, imagínese que quiere proyectar qué pasa con una inversión frente a una tasa de interés que varía desde el 1% al 10%, saltando de medio punto en medio punto. En lugar de escribir cada valor a mano como un pasante mal pagado, usted simplemente ejecutaría:

```
tasas = np.arange(0.01, 0.105, 0.005)
```

Fíjese que usamos 0.105 como límite final para que *NumPy* incluya el 0.10 (10%) en el resultado. Gracias a la *vectorización* que vimos antes, usted podría calcular el retorno de inversión para todas esas tasas al mismo tiempo con una sola línea de código. La eficiencia no es solo para las máquinas; es para que usted pueda terminar su trabajo antes y dedicarse a cosas más interesantes.

2.4.3 Función *np.linspace*

Si *np.arange* es una máquina que da pasos a tamaños fijos, *np.linspace* o *Linear space* es una máquina que rellena un espacio.

Básicamente, mientras que en *np.arange* generábamos datos definiendo el "salto", a *np.linspace* le decimos cuántos datos queremos dentro de un intervalo. Es decir, *np.linspace* tiene un inicio, un final y una cantidad de

elementos que deseamos repartir equitativamente entre ambos puntos. La estructura es:

```
np.linspace(inicio, fin, cantidad)
```

A diferencia de su primo *arange*, aquí hay una regla de cortesía importante, *np.linspace* utiliza intervalos cerrados $[inicio, fin]$, lo que significa que el número de inicio y el de fin sí están incluidos en el resultado. Véalo en código:

np.linspace

```python
import numpy as np

# Queremos generar 4 números repartidos entre 0 y 10.
linspace_basico = np.linspace(0, 10, 4) # nos retorna
    [ 0. , 3.33333333, 6.66666667, 10. ]
```

Usted, lector insistente, me va a preguntar para qué sirve esto. Pues bien, *np.linspace* es el mejor amigo de los gráficos. Si tenemos una función matemática y queremos dibujar una curva suave en lugar de una línea quebrada y espantosa, simplemente le pedimos a *linspace* que genere 100 o 1.000 puntos en el intervalo. A más puntos, más suave será la curva y más profesional se verá su trabajo.

2.4.4 Función *np.zeros*

La función *np.zeros* es como cuando te mudas solo y abres la heladera por primera vez: está totalmente vacía. Para los que vienen del mundo de la oficina, es como abrir una hoja de *Excel* nueva; no tiene datos aún, pero está lista para que usted empiece a rellenarla.

En el análisis de datos, muchas veces no tenemos los datos de inmediato, pero ya sabemos qué estructura van a tener. Esto nos permite reservar un lugar en la memoria de la computadora mediante la función:

```
np.zeros()
```

Esta función crea un *array* donde absolutamente todos los elementos son el número 0. Es la forma más común de "inicializar" un arreglo antes de llenarlo con información real.

Ejemplo 2.4.1

Imagine que nuestro chef de la boda quiere realizar una encuesta sobre la calidad de los platos. Antes de empezar a preguntar, arma una planilla con 1.000 filas vacías (es decir, con ceros). A medida que cada invitado responde, el chef anota el resultado; por detrás, *NumPy* borra el cero y escribe el nuevo dato.

Por lo tanto, *np.zeros()* es nuestra herramienta para crear esa "planilla en blanco". Para que funcione, necesita que usted le dé la *forma* (*shape*) dentro de los paréntesis: si quiere un vector, pase un solo número; si quiere una matriz, pase un par de números entre paréntesis (una tupla).

Véalo en código:

```
np.zeros

import numpy as np

# Creamos una lista de un vector con 5 ceros:
vector_vacio = np.zeros(5) # Retorna [0., 0., 0., 0., 0.]

# Creamos una matriz de 2 filas y 3 columnas
matriz_vacia = np.zeros((2, 3)) # OJO, es muy importante el doble parentesis, ya que el interno define la tupla y el de fuera la función.

# La matriz vacia retornaria:
# [[0., 0., 0.],
# [0., 0., 0.]]

# Si quiere enteros:
vector_enteros_vacios = np.zeros(4, dtype=int) # Retorna [0, 0, 0, 0]
```

2.4.5 Función *np.ones*

Si *np.zeros()* creaba una pizarra de ceros, *np.ones()* crea un *array* donde todos los elementos son, lógicamente, el número 1. Esta función es sumamente útil cuando necesitamos inicializar valores que luego servirán como base para multiplicaciones o para crear "máscaras" de datos.

Ejemplo 2.4.2

Imagine que tiene un vector de precios y quiere aumentar todos un 10% Una forma elegante de hacerlo sería crear un vector de unos del mismo tamaño, sumarle 0.10 (quedando todos en 1.10) y luego multiplicar ese vector por sus precios originales.

En cuanto a la sintaxis, funciona exactamente igual que *np.zeros()*, solicitando la "forma" (*shape*) deseada:

Vease en código:

Uso de np.ones

```python
import numpy as np

vector_unos = np.ones(3) # Retorna [1., 1., 1.]

matriz_unos = np.ones((2, 2))
# Retorna
# [[1., 1.],
# [1., 1.]]
```

2.4.6 Función *np.eye*

Esta función tiene un nombre que cobra sentido en inglés: la palabra *eye* suena exactamente igual que la letra "I", que en matemáticas representa a la *Matriz Identidad*. Lamentablemente, en español nos perdemos el juego de palabras, ya que "ojo" no tiene ninguna relación fonética con "identidad".

Una matriz identidad es una tabla donde todos los números son 0, excepto en la *diagonal principal* (aquella que recorre la matriz desde la esquina superior izquierda hasta la inferior derecha). En esa diagonal, mandan los unos. Matemáticamente, se ve así:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Más allá de la pizarra, está nuestro código. A diferencia de las funciones anteriores, *np.eye()* suele recibir un solo número, ya que las matrices identidad son, por definición, cuadradas (mismas filas que columnas). Veamos el código:

```
np.eye

import numpy as np

identidad = np.eye(3)

"""
Resultado:
[[1., 0., 0.],
[0., 1., 0.],
[0., 0., 1.]]
"""
```

En la aritmética común, el número 1 es el "elemento neutro" de la multiplicación: cualquier número multiplicado por 1 da el mismo número ($5 \times 1 = 5$). En el mundo de las matrices (Álgebra Lineal), la Matriz Identidad cumple esa misma función. Si multiplicas cualquier matriz por la Identidad, obtienes la misma matriz original.

¿Para qué nos sirve esto en análisis de datos? Muchas veces multiplicamos matrices para aplicar transformaciones, como rotar un gráfico, aplicar pesos a indicadores financieros o realizar proyecciones complejas donde necesitamos un "punto de partida" neutro.

2.5 Propiedades y estructura de los arrays

2.5.1 Dimensiones, forma, tamaño y tipo de datos

Dimensiones *ndim*

Las dimensiones son las que nos permiten entender cómo se organiza la información en el espacio (y en la memoria de su pobre computadora). La propiedad *.ndim* —o *number of dimensions* para el que todavía se pelea con el inglés— nos dice cuántas coordenadas necesita *NumPy* para encontrar un dato específico.

Básicamente, *.ndim* nos informa si nuestro objeto es:

- *Escalar (0 dimensiones):* Es un simple número suelto, como un 5. Imagine un punto en el vacío: no necesita dirección porque ya está ahí. No se mueva, no busque, solo es.
- *Vector (1 dimensión):* Es una línea. Necesita una coordenada (el índice) para saber en qué posición de la fila se encuentra el dato.

- *Matriz (2 dimensiones):* Es un plano o una tabla. Aquí ya necesita dos coordenadas: la fila y la columna. Si solo da una, *NumPy* le entregará una fila entera y se quedará esperando el resto.

- *Tensor (3 dimensiones):* Es un cubo. Aquí la cosa se pone tridimensional y necesita tres coordenadas: el piso (profundidad), la fila y la columna.

Saber el número de dimensiones es fundamental para no romper nada. Si usted intenta sumar un cubo con una línea, *NumPy* le lanzará un error de dimensiones en la cara; usar *.ndim* es su primera línea de defensa para depurar el código antes de que todo explote.

Veamos *.ndim* en acción:

Uso de .ndim en NumPy

```python
import numpy as np

# Uso de un vector simple
a = np.array([1, 2, 3])
print(a.ndim)  # Resultado: 1

# Uso de una matriz
b = np.array([
    [1, 2],
    [3, 4]
    ])
print(b.ndim)  # Resultado: 2

# Uso de un tensor o paquete de matrices
c = np.array([
    [[1], [2]],
    [[3], [4]]
    ])
print(c.ndim)  # Resultado: 3
```

Para aquellos que todavía no terminan de ”cazar” las dimensiones visualmente, hay un truco sucio pero infalible: cuente cuántos corchetes “[[” se abren al inicio del array. Si ve tres corchetes seguidos, felicidades, está ante un tensor de tres dimensiones. Si ve uno solo, es un vector. Es una regla simple, elegante y le ahorrará mirar la documentación más de lo estrictamente necesario.

Forma o *shape* (para los bilingües)

Si *.ndim* nos decía cuántas dimensiones tiene el objeto, *.shape* nos indica qué tan "grande" es cada una de ellas. Esta propiedad es, básicamente, una radiografía de sus datos.

Técnicamente, *.shape* nos devuelve una *tupla* (esos números entre paréntesis que no se pueden tocar), donde cada valor representa la cantidad de elementos en un eje específico.

Aunque parezca una propiedad inofensiva, la consultará más veces que su saldo bancario, ya que:

- Nos permite conocer la magnitud de los datos antes de procesarlos.
- Es el juez que valida operaciones matemáticas: si intenta sumar matrices con *shapes* distintos, la operación fallará miserablemente.
- Es fundamental para preparar los datos antes de enviarlos a librerías de gráficos como *Matplotlib*.

Véalo en esta demostración técnica:

Uso de .shape en NumPy

```python
import numpy as np

# Un Vector
vector = np.array([10, 20, 30, 40, 50])
print(vector.shape) # Resultado: (5,) es decir 5 elementos en una sola linea.

# Una Matriz (2D)
matriz = np.array([
    [1, 2, 3],
    [4, 5, 6]
    ])
print(matriz.shape) # Resultado: (2, 3) es decir  2 filas y 3 columnas.

# Un Cubo (3D)
cubo = np.zeros((2, 3, 4)) # Aqui utilizamos .zeros() para no morir escribiendo números
print(cubo.shape) # Resultado: (2, 3, 4) que son 2 matrices, cada una de 3 filas y 4 columnas.
```

Un detalle para su salud mental: note que en el vector el resultado es *(5,)*. Ese espacio vacío después de la coma está ahí para recordarle que solo hay una dimensión. No intente buscar el segundo número, no existe, es solo *NumPy* siendo técnicamente correcto (y un poco molesto).

Tamaño o *size*

Este es el último integrante de la "Santísima Trinidad" de la inspección. Mientras que los otros se preocupan por la estética y la estructura, *.size* es un contador pragmático: nos dice cuántos elementos hay en total en el *array*, sin importar cómo estén distribuidos en filas, columnas o dimensiones.

Matemáticamente, *.size* es el producto de todos los números que aparecen en *.shape*. Por ejemplo:

- En un vector con *.shape* (5,), el *.size* es 5.
- En una matriz con *.shape* (3,4), el *.size* es $3 \times 4 = 12$.
- En un tensor con *.shape* (2,2,2), el *.size* es $2 \times 2 \times 2 = 8$.

Usted, lector escéptico, se cuestionará: "¿Realmente necesito saber cuántos datos hay en total?". La respuesta es un rotundo sí, principalmente para no hacer explotar su sistema. El atributo *.size* nos ayuda a tener un control estricto de la memoria y es el juez supremo cuando queremos usar *.reshape()* (lo veremos en la siguiente subsección); usted no puede transformar una tabla de 12 elementos en una de $5 \times 55 \times 5$, porque los datos no aparecen por arte de magia.

En codigo (no muy diferente a los anteriores), esto se ve así:

Diferencia entre .size

```python
import numpy as np

# Creamos una matriz de 3 filas y 4 columnas

matriz = np.array([
[1, 1, 1, 1],
[2, 2, 2, 2],
[3, 3, 3, 3]
])

print(f"Total de elementos (.size): {matriz.size}") # Resultado: 12
```

Tamaño, forma y dimensión

Por si todavía tiene una ensalada de conceptos en la cabeza, aquí tiene el resumen. *.ndim* le dice en cuántas direcciones se mueven los datos; *.shape* le da el mapa exacto de esas direcciones; y *.size* le dice cuántos elementos hay en total.

La diferencia se observa mejor en el código:

Diferencia entre .ndim .shape y .size

```python
import numpy as np

# Creamos una matriz de 3 filas y 4 columnas

matriz = np.array([
[1, 1, 1, 1],
[2, 2, 2, 2],
[3, 3, 3, 3]
])

print(f"Dimensiones (.ndim): {matriz.ndim}")  # Resultado: 2
print(f"Forma (.shape): {matriz.shape}")      # Resultado: (3, 4)
print(f"Total de elementos (.size): {matriz.size}") # Resultado: 12
```

El metodo *.reshape()*

Seguramente usted, lector indignado, saltará a mi yugular diciendo: ”¡Esto no es un atributo, es un método! ¿Qué hace en esta sección?”. Pues bien, el autor soy yo y considero que explicar *.reshape()* aquí es mucho mejor que hacerlo más adelante, cuando usted probablemente ya haya olvidado qué es el *.shape*.

A diferencia de los atributos que vimos antes (que solo nos daban información), *.reshape()* es una herramienta activa. Nos permite cambiar la estructura de un *array* —sí, lo sé, suena a magia negra— *sin tocar ni uno solo de los datos que tiene dentro.*

Ejemplo 2.5.1

Imagine que tiene una tableta de chocolate con 12 cuadraditos dispuestos en una sola fila larga (nuestro chef se tomó un descanso, así que ahora comemos chocolate). *.reshape()* es el acto de romper esa fila y reacomodarla para que ahora tenga 3 filas de 4 cuadraditos. El chocolate es el mismo y el peso es idéntico, pero su forma ha cambiado para que entre en una caja distinta.

Para utilizar este método, el *.size* cobra más sentido que nunca. La regla de oro es inquebrantable: el número total de elementos debe ser exactamente el mismo antes y después de la transformación. Usted no puede generar datos de la nada ni puede hacer desaparecer cuadraditos de chocolate en el proceso; NumPy no tolera la magia contable.

Véalo en el código:

Uso de .reshape()

```python
import numpy as np

# Creamos un vector de 12 elementos (del 0 al 11)
vector = np.arange(12) # Forma actual: (12,) Tamaño: 12

# Lo transformamos en una matriz de 3 filas y 4 columnas
matriz = vector.reshape(3, 4)
print(matriz)
"""
Resultado:
[[ 0,  1,  2,  3],
[ 4,  5,  6,  7],
[ 8,  9, 10, 11]]
"""

print(matriz.shape) # Resultado: (3, 4)
```

El truco del comodín (-1): Si usted es un poco perezoso (como todo buen programador), NumPy tiene un regalo para usted. Si conoce cuántas columnas quiere, pero no tiene ganas de calcular cuántas filas saldrán, puede usar el **-1**. Es un mensaje que dice: "Calcula tú este número basándote en el *.size*".

El comodin -1 en reshape

```python
import numpy as np

vector = np.arange(12)
# Quiero 2 filas y no se cuantas columnas
nueva_forma = vector.reshape(2, -1) # Resultado:
    Matriz de (2, 6)
```

Tipos de datos *.dtype*

Si recuerda el inicio de este apunte, mencionamos que un *array* es estrictamente *homogéneo*, todos los elementos deben ser del mismo tipo. Pues bien, con *.dtype* (*data type*) podemos interrogar al arreglo para saber exactamente de qué estamos hablando.

> *Recordatorio*: Si usted combina tipos de datos, *NumPy* no se va a quejar, pero tomará decisiones por usted. Transformará todo al tipo de dato más "complejo" o flexible que encuentre. Si mete un *float* en una lista de enteros, todos serán *floats*. Si mete un *string*, prepárese, todos los números se convertirán en texto y perderán sus propiedades matemáticas.

El código no es muy complejo, vease:

.dtype para tipo de dato

```python
import numpy as np

# Creamos un array simple
array = np.array([1,2,3])
print(f'Tipo de dato: 'array.dtype) # devuelve: int
```

Es vital consultar *.dtype* antes de realizar operaciones matemáticas complejas; no querrá intentar calcular una raíz cuadrada sobre un *array* que *NumPy* decidió convertir en texto porque usted olvidó una comilla por ahí.

2.5.2 Conversión de tipos *.astype()*

Para finalizar este bloque, *NumPy* nos trae una función que nos permite forzar el tipo de dato de un *array* ya existente: *.astype()*.

Suponga que está realizando una investigación y se encuentra con millones de datos de coma flotante (*floats*). Usted determina que no tiene sentido cargar la memoria de su computadora con tanta precisión y cree que el error de conversión de decimal a entero no es significativo para su análisis. Pues bien, aquí es donde entra *.astype()* para salvarle las papas.

También, siempre que sea posible, usted puede convertir texto a números, siempre y cuando ese texto sea realmente un número disfrazado de *string*.

Vea el código en acción:

Uso de .astype()

```python
import numpy as np

# Creamos un array de decimales (floats)
precios = np.array([10.5, 20.9, 30.2])
print(precios.dtype) # Resultado: float64

# Queremos solo la parte entera
precios_enteros = precios.astype(int)
print(precios_enteros) # Resultado: [10, 20, 30]
# (Ojo: no redondea, corta el decimal)

# Convertir a texto
precios_texto = precios.astype(str)
print(precios_texto) # Resultado: ['10.5', '20.9',
    '30.2']

# Convertir de texto a float
precios_flotantes = precios_texto.astype(float)
print(precios_flotantes)
```

> **Advertencia:** Tenga mucho cuidado con el "hachazo". Al convertir de *float* a *int*, el valor 20.9 se convierte en 20, no en 21. Si su jefe le pregunta por qué faltan decimales en el presupuesto, no me eche la culpa a mí, yo le advertí que *NumPy* es eficiente, pero no necesariamente piadoso con el redondeo.

2.5.3 Indexación y slicing

Ahora que ya sabemos crear, deformar e interrogar a nuestros arreglos, llega el momento de aprender a operarlos.

La Indexación y el Slicing (o rebanado, para los que prefieren el castellano) son las herramientas que nos permiten extraer elementos específicos de un array.

> **Ejemplo 2.5.2**
>
> Imagine que su matriz es una pizza gigante: la indexación es elegir un pepperoni exacto, y el slicing es llevarse una porción completa (o media pizza, si tiene hambre de datos).

Indexación

La indexación es el sistema de direcciones que utiliza *NumPy*. Si un *array* es un estante lleno de cajas, la indexación es la etiqueta que tiene cada una para que sea fácil encontrarla, sacarla y usar lo que hay dentro.

La indexación puede ser vectoria, matricial o tensorial;

Una indexación vectorial (recuerde la fila de su secundaria), es como señalar a alguien de la fila. Para señalar a alguien solo se necesita saber su posición numérica o índice.

La indexcación matricial es cuando tenemos una tabla, recuerde el ejemplo de la reina en el tablero de agedrez, si usted la quisiera señalar deberia señalar la coordenada ij, es decir la posición de la fila y columna (no recuerdo exactamente la posición, pero seria por ejemplo $5C$).

Por ultimo la indexación tensorial es un poco más compleja, imagine un cubo de Rubik; es como señalar un color específico en una de las caras internas del cubo. Aquí ya necesita tres coordenadas (o más, si es que usted disfruta del sufrimiento matemático).

Para utilizar la indexación, simplemente colocamos los corchetes inmediatamente después del nombre del *array*: `array[indice]`.

Veamoslo actuar en código:

Indexación en NumPy

```python
import numpy as np

# --- Caso 1D --- #
edades = np.array([20, 25, 30, 35])
print(edades[0])  # Resultado: 20
print(edades[-1]) # Resultado: 35 (el último)

# --- Caso 2D --- #
```

```python
# Tabla de 2 filas y 3 columnas
ventas = np.array([
    [100, 120, 150], # Fila 0
    [200, 210, 250]  # Fila 1
    ])
dato = ventas[1, 0] # ventas del primer mes(columna), segundo año(fila)
print(dato) # Resultado: 200
```

Indexar no solo sirve para leer datos como si fuera un chisme; también sirve para escribir y corregir la realidad de su *array*.

Cambiando valores

```python
import numpy as np

precios = np.array([10.5, 20.0, 35.0])
# Queremos cambiar el segundo precio por 22 (ahora es 20)
precios[1] = 22.0

print(precios) # [10.5, 22.0, 35.0]
```

Si aún no termina de visualizarlo, considere este ejemplo culinario:

Ejemplo 2.5.3

El chef armo una lasaña de 9 porciones perfectas (3 filas y 3 columnas). Cada pedazo tiene un ingrediente secreto: uno tiene un borde quemado, otro mucha salsa, otro está fria, uno tiene demasiado queso, etc (Este chef no salio muy bueno ¡Se parece a mí!). Usted quiere elegir la de borde quemadito, pues no le parece nada cómico comer una porción con muchisima salsa, una porción fria o una que solo sabe a queso. Ademas le gusta esa textura crocante que deja el quemado. Pues usted cuando elije la porción esta haciendo una indexación (ademas de la ponderación) por lo tanto eligiria en el array lasaña el indice [i, j] donde se encuentra la lasañá con borde quemadito.

Indexación booleana

La indexación no termina con las coordenadas. El creador de *NumPy* sabía perfectamente que ni usted ni yo vamos a saber siempre en qué índice

exacto se encuentra la porción de borde quemadito del ejemplo 2.5.3. Por lo tanto, diseñó un ”colador inteligente”.

Se llama *booleana* porque solo entiende dos estados: *True* o *False* (*verdadero* o *falso*, si es que nunca se dignó a abrir un libro de inglés).

La indexación booleana funciona en tres pasos:

1. **La pregunta:** Le preguntamos al *array*: ”¿Dónde la temperatura de la lasaña superó los 100 grados?”.

2. **La máscara:** *NumPy* analiza cada elemento y crea un nuevo arreglo de verdaderos y falsos (*máscara*), marcando con *True* solo los lugares que cumplen la condición.

3. **El filtrado:** Le pedimos a *NumPy* que nos entregue el contenido original solo de los lugares donde la respuesta fue *True*.

Mas allá de su elección de la porcion de lasañá, en un análisis numérico serio (no más importante que elegir bien la porción de lasaña) esto nos ayuda a filtrar datos según lo que nos interesa: apartar valores atípicos (*outliers*), segmentar clientes por edad o encontrar acciones que superaron cierto precio.

En código:

Indexación Booleana paso a paso

```python
import numpy as np

edades = np.array([15, 25, 12, 40, 18])

# 1. La pregunta
# ¿Quiénes son mayores de edad (>= 18)?
# 2. La mascara o la busqueda de verdaderos y falsos
mascara = edades >= 18
print(mascara) # Resultado: [False,  True, False,
    True,  True]
# Los pasos 1 y dos estan relacionados

#2. filtrar el array
mayores = edades[mascara]
print(mayores) # Resultado: [25, 40, 18]

# --- Forma rápida (Todo en una línea) --- #

resultado_directo = edades[edades >= 18]
```

La indexación booleana también puede ser condicional. Si usted quiere filtros más complejos, de esos que requieren un ”esto Y aquello” o ”esto O lo otro”, podemos usar operadores lógicos.

Ahora, Dios sabrá por qué *NumPy* decidió no usar las palabras *and*, *or* y *not* de *Python* convencional (realmente es por una cuestión de eficiencia a nivel de bits), pero nos obliga a usar estos símbolos especiales:

- *AND* (Y): Se usa el símbolo “&” (ampersand).
- *OR* (O): Se usa el símbolo “|” (la barra vertical o *pipe*).
- *NOT* (NO): Se usa el símbolo “~” (la virgulilla de la ”ñ”).

Veamos esto en código:

Filtros complejos

```python

precios = np.array([10, 50, 100, 150, 200])
# Precios entre 60 y 180
# Cada condición debe ir entre paréntesis ( )
filtro = (precios > 60) & (precios < 180)

print(precios[filtro]) # Resultado: [100, 150]
```

Slicing

Si la indexación era señalar la porción de lasaña con el dedo, el *slicing* (o rebanado) es meter el cuchillo y llevarse una sección entera utilizando el operador de los dos puntos (:).

El slicing consiste de la siguiente sintaxis o estructura básica:

```
array([Inicio:Fin:Paso])
```

> El fin al igual que en *np.arange* no considera el valor final, es decir nunca lo toca (recuerde el intervalo semiabierto).

El slicing vectorial es el más intuitivo. Imagine un *array* de 6 elementos ($[0, 1, 2, 3, 4, 5]$). Usted puede realizar los siguientes cortes:

- array[1:4]: Te da los elementos en las posiciones 1, 2 y 3. (El 4 queda fuera).

- array[:3]: Los primeros tres (del inicio hasta el índice 2).
- array[3:]: Desde el índice 3 hasta el final.
- array[::2]: Todo el array, pero saltando de dos en dos (posiciones 0, 2, 4).

En el caso del slicing matricial, tenemos filas y columnas, por lo tanto usariamos comas para separar los cortes de cada dimensión:

```
array([corte_filas,corte_columnas: ...])
```

Se pueden realizar por ejemplo estos slicing:

- Seleccionar una fila entera: array[1, :] (Fila índice 1, todas las columnas).
- Seleccionar una columna entera: array[:, 0] (Todas las filas, columna índice 0).
- Seleccionar una sub-tabla: array[0:2, 1:3] (Filas 0 y 1; columnas 1 y 2).

No nos vamos a meter a tensores, ya que nadie quiere perder la cabeza (ese nadie soy yo).

Veamos el slicing en acción en codigo:

Slicing en NumPy

```python

import numpy as np

# Creamos una matriz de 4x4 (puedes imaginar meses vs productos)
datos = np.arange(16).reshape(4, 4)
"""
[[ 0,  1,  2,  3],
[ 4,  5,  6,  7],
[ 8,  9, 10, 11],
[12, 13, 14, 15]]
"""

# 1. Extraemos la segunda fila completa
fila_2 = datos[1, :] # [4, 5, 6, 7]

```

```
# 2. Extraer la última columna completa
ultima_col = datos[:, -1] # [3, 7, 11, 15]

#3. Extraer el cuadrado central (2x2)
centro = datos[1:3, 1:3]
"""
[[ 5,  6],
[ 9, 10]]
"""
```

> *Atención*: El *slicing* en *NumPy* no crea un *array* nuevo, crea una "vista" del original. Esto significa que si usted modifica la rebanada, *también modifica el array original*. Si quiere un objeto independiente para hacer sus experimentos sin arruinar la fuente, use *.copy()*.

2.6 Operaciones matemáticas

Además de las operaciones vectoriales básicas que ya vimos, existen las funciones universales, mejor conocidas como *ufuncs*. Lo más importante que debe saber es que estas funciones se aplican de manera vectorizada.

Si usted quisiera sacar la raíz cuadrada de 100 datos a mano, tendría que sacar la calculadora y empezar a pelearse con cada número. Pero como usted no quiere entregar su análisis un semestre más tarde, utiliza *NumPy*, que aplica la operación a todo el vector, matriz o tensor elemento por elemento sin perder ni un ápice de eficiencia. ¡Si no fuera así, usaríamos la librería *math* y nos iríamos a dormir una siesta mientras la computadora procesa!.

2.6.1 Raiz cuadrada y exponencial

Estas son las funciones "base" para una infinidad de cálculos estadísticos, financieros y científicos.

- Raiz cuadrada: *np.sqrt*, que calcúla $\sqrt{x}$ para cada elemento.
- Exponencial: *np.exp*: calcula e^x (donde e es el numero de Euler). Esta función es fundamental para calcular intereses compuestos, crecimientos poblacionales o para cuando su código decide crecer exponencialmente en errores. Podría dedicar toda una sección a explicar la belleza dee^x, pero lamentablemente este no es un libro de álgebra y ambos tenemos cosas mejores que hacer.

Veamos estas operaciones en acción en su hábitat natural:

Raiz y Exponencial

```python

import numpy as np

arr = np.array([1, 4, 9, 16])

# Raíz cuadrada
raices = np.sqrt(arr) # [1., 2., 3., 4.]

# Exponencial (e^x)
exp_valores = np.exp(np.array([0, 1, 2])) # [1.,
    2.718..., 7.389...]
```

Observe que los resultados son siempre *floats* (decimales). NumPy sabe que las raíces y las exponenciales rara vez son exactas, así que se prepara para la imprecisión de la vida real desde el principio.

2.6.2 Logaritmo Natural

Si en matemáticas e^x nos dice cuánto creció una población en un tiempo determinado, el logaritmo natural ($ln(x)$) nos dice cuánto tiempo tuvo que pasar para alcanzar ese crecimiento.

Básicamente, el logaritmo natural es como un domador de monstruos numéricos. Se usa principalmente para:

- *Normalizar escalas:* Si tiene datos que van de 1 a 1,000,000 (como seguidores en TikTok o la deuda externa), graficarlos es un caos visual. El logaritmo "aplasta" los números gigantes y "estira" los pequeños, haciendo que todo quepa elegantemente en una gráfica.

- *Linealizar relaciones:* Convierte curvas exponenciales complejas en líneas rectas, lo cual es oro puro para entrenar algoritmos de Inteligencia Artificial sin que su computadora explote en el intento.

- *Interés compuesto:* Fundamental para calcular cuánto tiempo tardará en ser millonario si decide invertir sus ahorros en lugar de gastárselos en café.

Imagine que se generó una colonia de bacterias que crece de forma masiva (e^x) y usted quiere saber qué tan rápido fue ese crecimiento. Utilizando *np.log()*, podemos estimar esos tiempos de la siguiente manera:

Logaritmo natural

```python
import numpy as np

# Cantidad de bacterias en diferentes muestras
bacterias = np.array([1, 10, 100, 1000])

# Aplicamos el logaritmo para "suavizar" los datos
tiempos_estimados = np.log(bacterias)

print(tiempos_estimados)
# Resultado: [0. , 2.30258509, 4.60517019, 6.90775528]
```

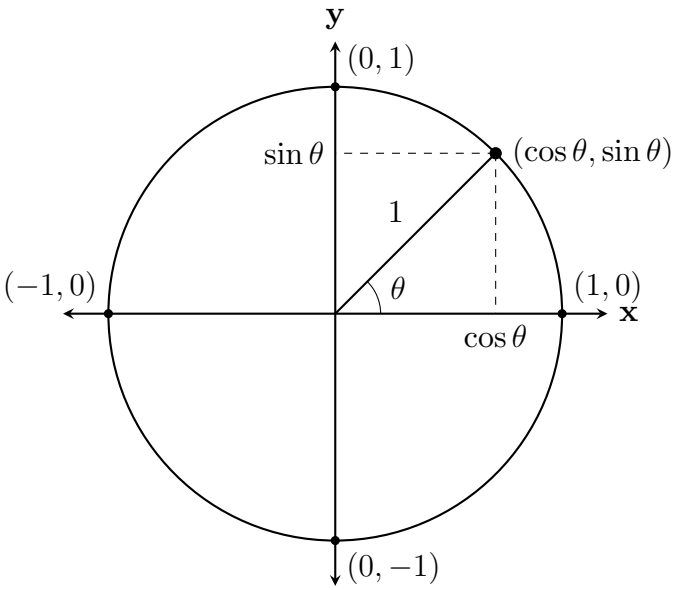
Observe cómo pasamos de una escala que saltaba de 10 en 10 (multiplicando) a una escala lineal que avanza de forma constante. Eso es *NumPy* poniendo orden en el caos.

> **Advertencia:** El logaritmo natural es una función muy útil, pero tiene un temperamento difícil: no acepta números negativos ni el cero. Si intenta calcular *np.log(0)*, *NumPy* le devolverá un $-\infty$ (*infinito negativo*) y un aviso de que algo está muy mal en su vida (o al menos en sus datos).

2.6.3 Funciones trigonométricas

Imagine que tiene un asador de pollos giratorio. Las funciones trigonométricas no son más que las coordenadas que nos dicen dónde está el pollo en cada momento mientras da vueltas en un círculo unitario.

Imagine un círculo cuyo radio es exactamente 1. El pollo empieza a la derecha y gira en sentido contrario a las agujas del reloj. El ángulo (θ) es qué tanto ha girado el asador.



Hay tres funciones principales:

- Seno sin: Es la altura del pollo. Cuando el ángulo es 0°, la altura es 0. Cuando llega arriba (90°), la altura es 1.
- Coseno cos: Es la distancia horizontal desde el centro. Cuando el ángulo es 0°, el pollo está todo a la derecha (1). A los 90°, está justo en el centro horizontal (0).
- Tangente tan: Es la relación entre la altura y la distancia. Imagina la sombra que proyecta el pollo en una pared vertical lejana.

Si usted no es ingeniero ni arquitecto, podría pensar que esto no tiene utilidad en el análisis de datos. Se equivoca: la trigonometría está en todas partes, desde el procesamiento de audio y música, hasta el análisis de ciclos eléctricos, señales de radio y física de videojuegos.

> En *NumPy*, las funciones esperan que les hables en Radianes, no en grados. $180° = \pi$ radianes (aprox. 3.14159).

Veamos cómo interrogar al pollo en código:

Funciones trigonométricas

```python
import numpy as np

# Queremos saber la altura del pollo a los 90 grados
```

```python
angulo_grados = 90
angulo_rad = np.radians(angulo_grados) # Convertimos a radianes

altura = np.sin(angulo_rad)
distancia = np.cos(angulo_rad)

print(f"A los 90°, la altura (Seno) es: {altura}") # Debería ser 1.0
print(f"A los 90°, la distancia (Coseno) es: {distancia}") # Debería ser casi 0
```

Como habrá notado, el coseno de 90° no dio exactamente 0, sino un número absurdamente pequeño (6.12×10^{-17}). Esto sucede porque las computadoras tienen limitaciones de precisión decimal. No se asuste; para nosotros, eso es cero.

Existen miles de funciones como *np.radians* o *np.degrees* que no vamos a abarcar aquí. Si pretendiera explicar cada función de la librería, este apunte tendría más hojas que la documentación oficial y usted se quedaría sin espacio en su disco duro. Cuando necesite algo muy específico, no dude en consultar la documentación de *NumPy*.